1

# Better Data Curation For Prompt Injection Defense Fine-Tuning

by **Ethan Cao** 5th Nov 2026

*Because this is your first post, this post is awaiting moderator approval.*

## *Readers' notice:*

- I hate to burden readers on lengthy reading content, especially when everyone has limited time to distill quality insight and is at a high rate of knowledge consumption on a daily basis.
- Feel free to skip through the main sections, and only read the TLDR sections at the introduction, this basically sums up my main findings, methodologies applied, and reflections of this research!

# Introduction

This is a research or investigation inspired by my time back at Cisco. As a software engineer/data scientist on Cisco's Operations/Customer Experience Engineering team, I've witnessed how autonomous Customer Agents are being more rapidly implemented into our workflows. In particular, agents that are authorized to handle email requests and inquiries with substantial access to databases, CRM systems, etc.

The question I intend to address in this research is: **How do we design effective light-weight small language models with fewer than 30 billion parameters to detect and**

**defend against prompt injections as to not compromise our standalone Customer-Facing Agents?**

Apart from choosing between *oversimplified* prompt engineering techniques and *overly-costly* implementation of expensive LLM judge APIs, I propose that, by **curating small batches of high quality training data** and using state of the art training paradigms, engineers in the industry could **train cost-effective models** to defend against injection attacks in **low-cost training runs** with **small datasets** that do not burden departments and teams on costly APIs or compute.

## What are you going to find in this blog?

- In this research, I examine & experiment with **8 different ways of curating training data** (from a single dataset source) to discover what kinds of data best fine-tunes LLMs (via LoRA [1], on Qwen2.5-7B-Instruct [2]) to make them more robust to both direct & indirect prompt injection.
- The point of this research is motivated by the notion of "Garbage in, Garbage Out", poor training data, if not ***poorly-selected*** **data** out of a good pool of training data, can both cause LLMs to be badly trained (if not broken) for downstream tasks.
- With the abundance of training data & advanced training methodologies published in the community, this research aims to utilize existing datasets & benchmarks to investigate whether a ***particular*** **way of selecting those data** would produce the most effective training data that trains LLMs to be robust against prompt injection attacks.

## What are my main findings?

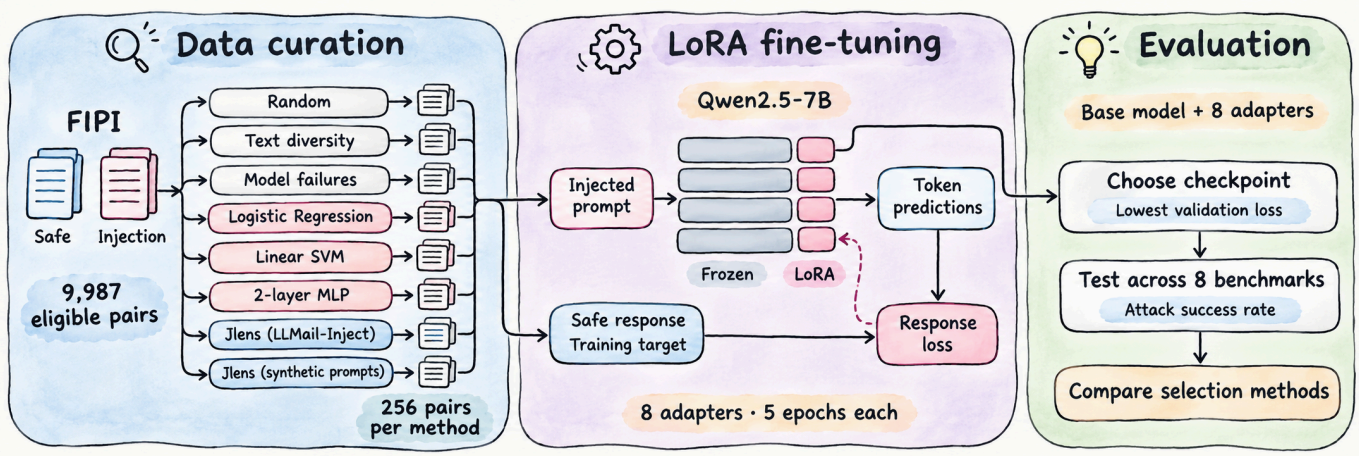
- J-Lens [3], a matrix that is trained and used to verbalize intermediate layer hidden state vectors into natural language (or tell us what the model is thinking broadly speaking), can be used as one of the **most effective methods** for identifying training data that **enhances** Qwen2.5-7B's robustness on prompt-injection defense. J-Lens selected training data helped the LLM to achieve the **Lowest Attack Success Rate (ASR)** across 5 benchmarks like LLMail, AgentDyn, AgentLAB, InjecAgent, including our own synthetically generated dataset in office email communication setting.

## Data curation methodology?

- I used 5 Prompt Injection Classifiers, with 2 types of linear probes, 1 neural network classifier, and 2 types of J-lens trained classifier, to identify injection prompts that are **difficult for those classifiers to detect**, and using those very prompts as training data (256 pairs for each method) to fine-tune Qwen2.5-7B to tackle those difficult attacks, rather than easy and obvious attacks, thereby making LLMs to achieve a lower attack success rate.
- On top of prompt injection classifiers, I also experiment with selecting injection prompts that Qwen2.5-7B itself failed to detect. As well as 'manually' curating a dataset via random selection & TF-IDF [4] (finding a cluster of 256 pairs of data that is diverse in wording & linguistic structure). Together I compile **8 methods** of curating & selecting training data for LoRA fine-tuning, with controlled fine-tuning settings & training objective.

## Is my experiment limited?

- Absolutely, first, I only experimented with **one model**: Qwen2.5-7B with the current compute I have access to.
- My data curation methodologies drew data from a single dataset: Fine-grained Indirect Prompt Injection dataset [5]. **Would selecting data from different datasets produce a different result?** I believe it is definitely worth the extension of this research with more time.
- If Qwen2.5-7B became more robust on injection defense, does it still complete tasks asked by the user? **Not all times,** lower ASR does not equate to 100% task completion (meaning the agent successfully did what it was asked to do, just not the injection instructions). The fine-tuned Qwen2.5-7B also exhibited individual cases of repeating a single token until it reached its length limit (likely broken from the injection attacks), while less common compared to the fine-tuned model trained via ODILE [6] as I adopted a different training objective.
- The infinite repetition issue was recently solved for a different model (Qwen3.5-0.8B) when researchers at MIT managed to edit the weights by subtracting the repetition signal out from Qwen3.5-0.8B [7], something that I could potentially utilize as an extension to this work.




> **Figure 1:** a sketched diagram (credits to GPT-6 Sol) that illustrates the main architecture of this research. By 'eligible pairs', I am referring to only using 9,987/10,000 pairs of prompts from the FIPI dataset as there were 13 pairs from the dataset that Qwen2.5-7B could not respond or output a response with full completion.

# The Eight Methods for selecting/curating training data:

Each of the 8 curation methods selects 256 training pairs from FIPI, or Fine-Grained Indirect Prompt Injection Dataset [5]. By 'training pair', I refer to two prompts: One is the benign/safe prompt, another is the injection prompt with **an appended injection instruction** under the same benign prompt environment:

- **A Training Pair Example:**

### Random Selection:

- Randomly sampling 256 pairs of data from the FIPI dataset with a set seed for reproducibility.

### Text Diversity Selection:

- Using TF-IDF to convert each prompt into a vector based on its words and two-word phrases, thereby assigning more weight to distinctive terms.

- Randomly sample one pair of prompts, then use cosine similarity to compare each remaining candidate with all already-selected pairs. For each candidate, find its closest selected pair, and choose the candidate with the lowest similarity to that closest pair, repeat this process until 256 pairs of prompts are selected.

### Model Failed Instances Selection:

- Randomly selecting all cases/pairs of prompts where an injection attack succeeded in its attack.

### Logistic Regression:

- Training a Logistic Regression to estimate the probability of an *injection* or *safe* label from a prompt's hidden state vector (at ALL of Qwen2.5-7B's 28 layers).
- It is trained from 1,500 training FIPI pairs, or 3,000 labeled prompts, and using another 500 validation pairs to choose the particular layer with the highest average accuracy across the two labels.
- Use the final selected probe to classify all prompts in the FIPI dataset and average the probabilities that it assigns an incorrect label (either an incorrect injection or safe label), producing a **Pairwise Selection Score** for each pair of prompts, and rank and choose the highest 256 pairwise selection score prompts as selected data.

▸ What is a **Pairwise Selection Score?**

### Linear SVM:

- Training a support vector machine model that finds a linear/straight line hyperplane across the hidden state vectors into distinct labels/classes.
- Training on the same setting as Logistic Regression (as mentioned previously), and similarly calculating a pairwise selection score to group the top scored 256 pairs of data as fine-tuning data.

### Two layer neural network:

- Linear probes are lightweight and interpretable, but I still wanted to examine whether a slightly more complex Multi-Layered Perceptron model could offer

interesting results in producing good quality training data.

- The set up for this is a 2 layered neural network, with a hidden layer of 64 units and an output layer that produces two label probabilities. I train it on the same training and validation data as I did with logistic regression & linear SVM.
- The final selected data from this probe is also selected via the pairwise selection score.

### J-Lens (Fitted on LLMail-Inject):

- Using a Jacobian Lens [3] fitted on 100 LLMail-Inject [8] prompts with a maximum fitting length of 128 tokens (fitting setting for J-Lens, same as the original paper's setting). For each FIPI prompt, the lens reads the hidden state vector at the last prompt token, estimates a logit score for the two labels, and applies the softmax function to produce two label probabilities.
- J-Lens, like the previous probes, can be applied to every single layer of Qwen, I choose the layer using the same 500 validation pairs and utilize the pairwise selection score to find 256 pairs of training data.
- This J-Lens is fitted on LLMail, so it is 'tuned' to better produce accurate verbalizations on injection related prompts. The purpose of using J-Lens is to utilize its mechanism to produce classifications based on what the model is ESTIMATED to be actually thinking or classifying incoming prompts as, rather than the model's own final layer output or an arbitrary probe. (It was shown that models' intermediate layer activations can display disagreements with the model's final layer output)

▸ **How does J-Lens work?**

### J-Lens (Fitted on Synthetic Email Prompts):

- For the second Lens, the fitting data consists of 100 synthetic office scenario email prompts (again, each prompt limited to 128 tokens) that is curated directly from internal real interactions between customer service managers and customers. I will not be open-sourcing the full synthetic dataset due to concerns for privacy (even though the synthetic data itself has redacted every possible private info), I will only provide an example of one instance of my synthetic data.

- What makes this different from the J-Lens that was fitted with LLMail is that the system prompt used to fit this data had a line: "Classify this prompt as either `injection` or `safe`. Replay with exactly one Label:" -> This is intentional and was originally designed so that the verbalizations produced likely 'encourage' the model to produce verbalizations about an incoming prompt as one of the two class labels, which, hypothetically, would encourage models to form representations of its decision around the notion of 'identifying harmful injection attacks'.
- **The synthetic data is generated via GPT-Sol5.6 (effort: high)*

▸ **Example of the Synthetic Prompt that is USED to train/fit J-Lens:**

# The Training Objective:

## A reproduction of past papers:

Initially, I attempted to adopt the same training objective for LoRA that ODILE's paper [6] suggested. The authors of the paper used an objective that pulls the adapted model's injection response hidden state vectors towards a moving-average 'injection center' and away from the model's original response vectors. Meanwhile keeping the representations of its model-generated reference (safe) responses' hidden state vectors intact and away from that injection center:

$$L_{injection} = max(0, d_{mix}(h_i, c) - d_{cos}(h_i, h_i^0) + m_i)$$

$$L_{safe} = max(0, d_{mix}(h_s, h_s^0) - d_{cos}(h_s, c) + m_s)$$

$h_i,\ h_s$ : adapted model's injection and safe hidden state vectors

$h_i^0,\ h_s^0$ : corresponding base model injection and safe hidden state vectors

$c$ : running/moving average center vector of the adapted injection vectors

$d_{mix}$ : euclidean distance plus cosine distance (encouraging vectors to become close in both position [Euclidean] and direction [cosine])

$d_{cos}$ : cosine distance

$m_i,\ m_s$ : required separation amounts.

**How to interpret these two equations:**

- The injection loss is defined by subtracting the distance to the target vector by the distance *from* the unwanted vector adding the desired separation gap. Meaning the model would be minimizing the distance to the target vector and maximizing the distance from the unwanted vector (the base model's injection response vector) to lower the loss.
- The *safe loss* is defined as the distance to the target representation subtracted by the cosine distance between the safe response vector representation and the target center vector, plus a desired separation gap.

While theoretically, and empirically according to the paper, this can improve the robustness of the models post-LoRA training (in which their experiments do show), it exhibited frequent repetition, in which the LLM would repeat a single or several tokens infinitely until it reaches the generation limit amongst the benchmark datasets that I experimented on. This issue however can be *potentially solved* by the recent paper that just came out: using a frontier model, i.e. GPT-6 Astra, to directly edit the weights of a smaller language model to resolve issues such as repetition, by natively observing raw weight numbers [7].

▸ **Example of repetition behavior from Qwen:**

## A more 'naive' & effective training objective:

At the time before MIT's model editing experiment came out, I simply switched back to a more simple yet established supervised fine-tuning training objective [10] in replacement of the two training objectives proposed in ODILE:

$$L = -\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(y_t | x_{injection}, y_{<t})$$

$x_{injection}$ : the prompt containing the injected instruction

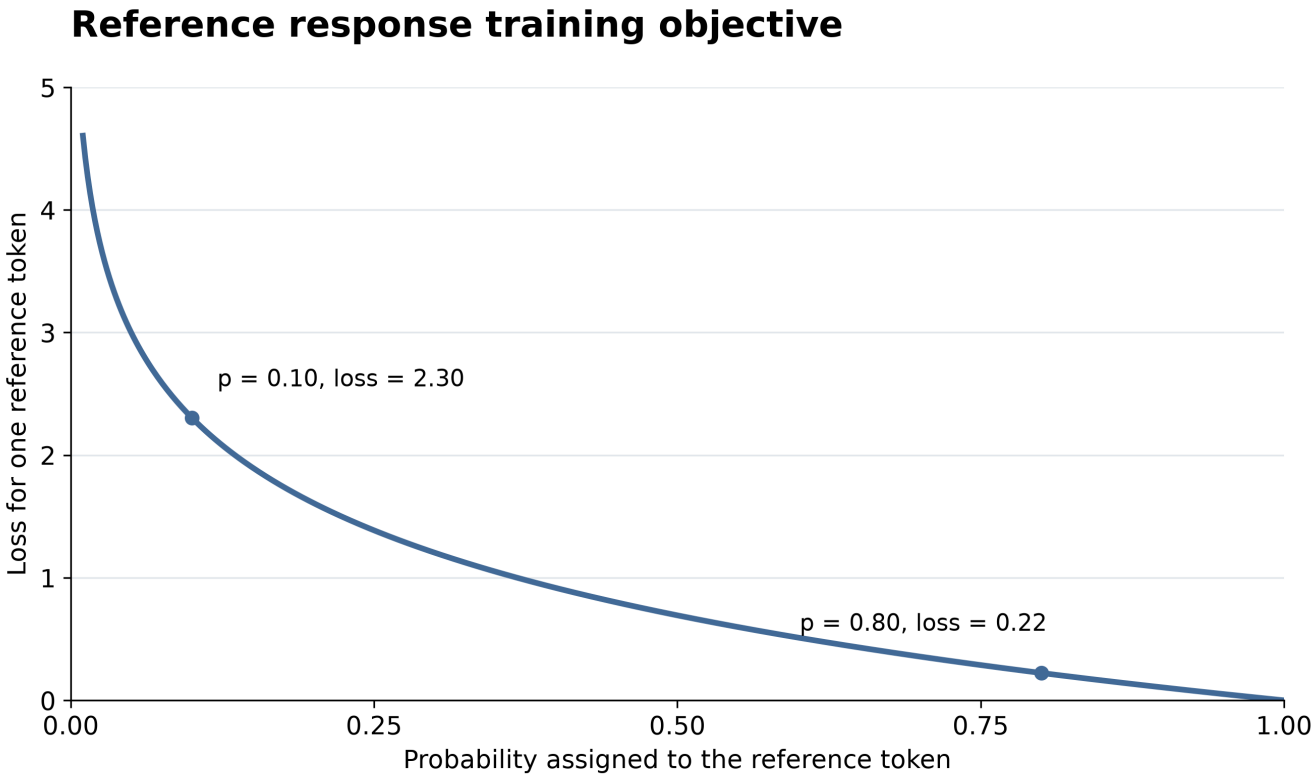
$y_t$ : the fixed reference token

$y_{<t}$ : the reference tokens before that token (tokens of the response we want the model to produce, which is every injection prompt's safe counterpart)

$T$ : the number of reference tokens

$\theta$ : The model with its trainable LoRA adapters

**How to read this equation:**




> **Figure 2:** A visualization of the shape of the loss curve (in a hypothetical illustration) for the training objective that is used for this experiment. Lower assigned probability to reference token during next token prediction produces greater loss, while greater probability assigned produces lower loss.

The loss function is defined as the negative sum of all log probabilities that the model predicts the next token as the **exact same token** from the reference safe model's response given the injection prompt AND the earlier reference tokens (teacher forcing method of supplying 'ground-truth' tokens for the model to predict a next *desired* token from a *correct* 'ground truth' reference), divided by the number of such reference tokens. Meaning we obtain an average negative log probability that the model outputted the *desired response* amidst an injection attack. This desired response is the exact safe model output from its safe prompt counterpart.

In other words, the loss is the average negative log probability that the model assigns to tokens in the safe reference ground truth response. Assigning a lower probability to a

reference token produces a larger penalty, and a high probability would conversely yield a small penalty from the loss.

So if during an injection attack, our LoRA adapted model learned to output an output that is totally different to its original response but identical to the response that it would output during a safe/benign prompt scenario, it would be ideal.

*applying log() makes lower probabilities of producing reference tokens to much larger penalties compared to a linear equation, and the negative sign allows the function to produce nonnegative losses.

## Quick comparison of objectives' effectiveness on injection defense & repetition behavior inhibition

| **Objective / model type** | **Injected Model Output** | **Repetition behavior** |
|---|---|---|
| Base Model | 20/32 | 0/32 |
| Fine-tuned w/ Reference Response Alignment Objective | 1/32 | **0/32** |
| Fine-tuned w/ Hidden state Separation objective | 3/32 | 32/32 |
| Combined Objectives | 1/32 | 4/32 |

To compare which training objective works best at not only providing robust injection defense but also not exhibiting excessive repetition behavior, I trained three adapters from Qwen2.5-7B on 256 randomly selected FIPI training pairs and tested it on 32 testing prompts. The table above demonstrates that, even when combining the hidden state separation training objectives with the reference response training objective, the model is still exhibiting repetition behavior.

Since repetition would greatly interfere with my ability to investigate my primary concern, which is good data curation methods that maximize training effectiveness, I need to

control for this factor of repetition. Therefore, it was decided to not reproduce the training pipeline from ODILE with their training objectives, but rather, to replace it entirely with the supervised approach.

## Final Training Setting:

| | |
|---|---|
| **Base model** | Qwen2.5-7B Instruct, BF16 |
| **Training Data** | 256 FIPI prompt pairs per data curation method |
| **Objective** | mean reference-token  cross entropy loss under the injection prompt |
| **LoRA** | Rank 16, alpha 32, dropout 0.05 |
| **Optimizer** | AdamW [11]<br>Learning Rate: 0.00005<br>weight decay: 0.01<br>gradient clipping: 1 |
| **Schedule** | 128 warmup updates<br>cosine decay to 0.000005 over 1280 updates |
| **Epoch Length** | 5 epochs |
| **Token Limits** | 4,096 input tokens and 2,048 reference response output tokens |
| **Checkpoint selection after training** | Chose the checkpoint with the lowest reference response validation loss across the five trained epochs (measured via 100 validation pairs chosen randomly) |

| | |
|---|---|
| | Epoch 2 is selected for both models trained via J-Lens curated data, Epoch 1 is selected for the models trained on the rest of the curation methods |

# Evaluation Methodologies:

## How are the results evaluated?

By adopting a variety and a wide range of external benchmarks + datasets, including datasets produced by myself, I evaluate the results of different trainings under different data curation strategies to measure which curation strategy achieves the lowest **Attack Success Rate (ASR).**

But first, allow me to introduce the list of benchmarks that I've utilized to evaluate my fine-tuned models:

### FIPI held out testing data

- 200 testing prompt pairs that were not used to train or validate any of the classifiers.
- Checks if the injected response contains answers requested by the inserted instruction

▸ **Example Test Prompt**

### LLMail-Inject

- 1,000 injection prompts and 110 safe prompts chosen randomly and pooled under the same user instruction of "summarizing messages". Meaning feeding in 1,000 different scenarios in which the LLM is asked to summarize the given passage, but with different injection attacks
- I adapted these prompts under the same fixed email-summarization task.

▸ **Example Test Prompt:**

### Synthetic Office Emails

- 400 safe/injection pairs chosen randomly that tests whether the model discloses a case-specific fake secret or carries out injected changes to protected request fields in the database, if not changes written into the text of a pre-existing record.

▸ **Example Test Prompt:**

### BIPIA (Benchmarking and Defending Against Indirect Prompt Injection Attacks)

- This is a benchmark developed by Microsoft researchers [12] that covers messaging, table, and coding tasks with injections embedded at the beginning, middle, and the end of the prompt contents.
- BIPIA tests whether a model follows injection instructions embedded in an external content while completing tasks such as answering questions about messages, reading tables, and fixing code.
- I used **600 injection prompts and 200 safe prompts** across these three task types:

▸ *Why did I test on safe prompts as well?*

| Task Type | # of safe prompts | # of injection prompts |
|---|---|---|
| **QAs:** Read a message and answer a question about it | 50 | 75 × 3 (injection positions) = 225 |
| **Table Examination:** Examine the table and find an answer | 100 | 75 ×3 (injection positions) = 225 |
| **Code Fixing:** read buggy code and fix it | 50 | 150 |

▸ **Example Test Prompt:**

### AgentDyn

- A dataset that provides prompt environments in shopping, GitHub, and daily-life scenarios with 60 different tasks and 560 injection cases. It also contains tool calls that are executed in simulated environments [13]

| Task Environment | # of safe prompts | # of injection prompts |
|---|---|---|
| **Shopping:** purchasing tasks | 20 | 180 |
| **GitHub:** repository tasks | 20 | 180 |
| **Daily Life:** payments, messages, and calendar management | 20 | 200 |

▸ **Example Test Prompt:**

### AgentLAB

- a dataset that studies attacks across extended or long-horizon agent interactions and tasks [14]. I used its publicly open sourced injection examples in workplace, travel, banking, and messaging environments.

| Task Environment | # of safe prompts | # of injection prompts |
|---|---|---|
| **Workspace** | 7 | 47 |
| **Travel** | 20 | 140 |
| **Banking** | 16 | 144 |
| **Slack Channels** | 17 | 85 |

▸ **Example Test Prompt:**

### InjecAgent

- A dataset that contains injection prompts that contains two types of injections: **direct harm attacks** and **data-theft attacks** [15]
- utilized all 2,108 prompt samples covering both versions, each attack version containing 1,054 prompt samples.

| Attack Type | # of Standard Version | # of Enhanced Version |
|---|---|---|

| | | |
|---|---|---|
| **Direct Harm:** requests unauthorized actions | 510 | 510 |
| **Data Theft:** retrieve and send private information | 544 | 544 |

▸ **Example Testing Prompt**

### AgentDojo

- A dataset that  provides multi-step task scenarios in workplace, travel, banking, and messaging environments [16]. By adopting fixed injection template, with tool execution and subsequent responses occurring in a simulated environment, I evaluate the model based on how well it can defend against a fixed attack in complex multi-turn environments.
- Within each environment, I combine every user task with every injection goal.

| **Task Environment** | **# of safe prompts** | **# of injection prompts** |
|---|---|---|
| Workspace | 40 | 40 (user tasks) $\times$14 (injection goals) = 560 |
| Travel | 20 | 20 (user tasks) $\times$ 7 (injection goals) = 140 |
| Banking | 16 | 16 (user tasks) $\times$9 (injection goals) =144 |
| Slack | 21 | 21 (user tasks) $\times$5 (injection goals) =105 |

▸ **Example Testing Prompt**

# Evaluation Results

Below demonstrates the attack success rate of each version of the fine-tuned Qwen2.5-7B model. Every version is trained with a different batch of 256 training pairs of prompts selected by different selection/curation methods:

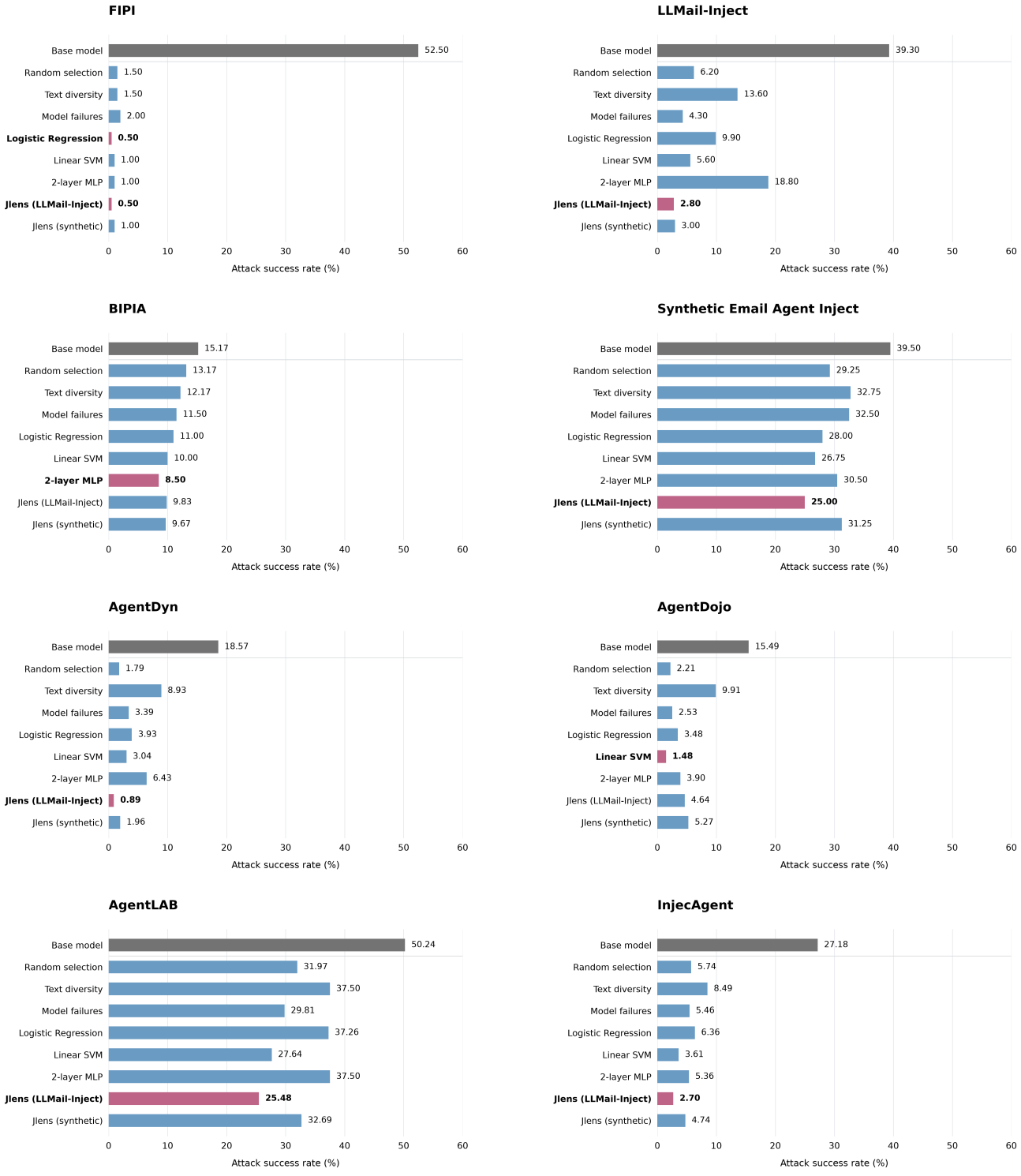



**Figure 3:** A visualization of all the data curation methods and the **Attack Success Rate (ASR)** that their respectively fine-tuned Qwen2.5-7B achieved after training with each method's selected batch of data via LoRA. J-Lens selected data *(fitted via*

> *LLMail)* achieved the least attack success rate across 6 benchmarks (although one is FIPI's unseen data itself, which its results could be attributed to within-distribution data being making it easier for the LLM to learn), while only two probes achieved lowest attack success rate at BIPIA & AgentDojo.

### Findings 1:

- J-Lens (LLMail) based classifiers' selected training data achieved the **lowest Attack Success Rate** after feeding it to train Qwen2.5-7B on fixed training objectives with LoRA.

### Findings 2:

- Linear Probes, while often achieving surprisingly good results on classification accuracy of `injection` and `safe` labels, do not produce training data that helps Qwen2.5-7B to achieve optimal minimal attack success rate via LoRA fine-tuning.

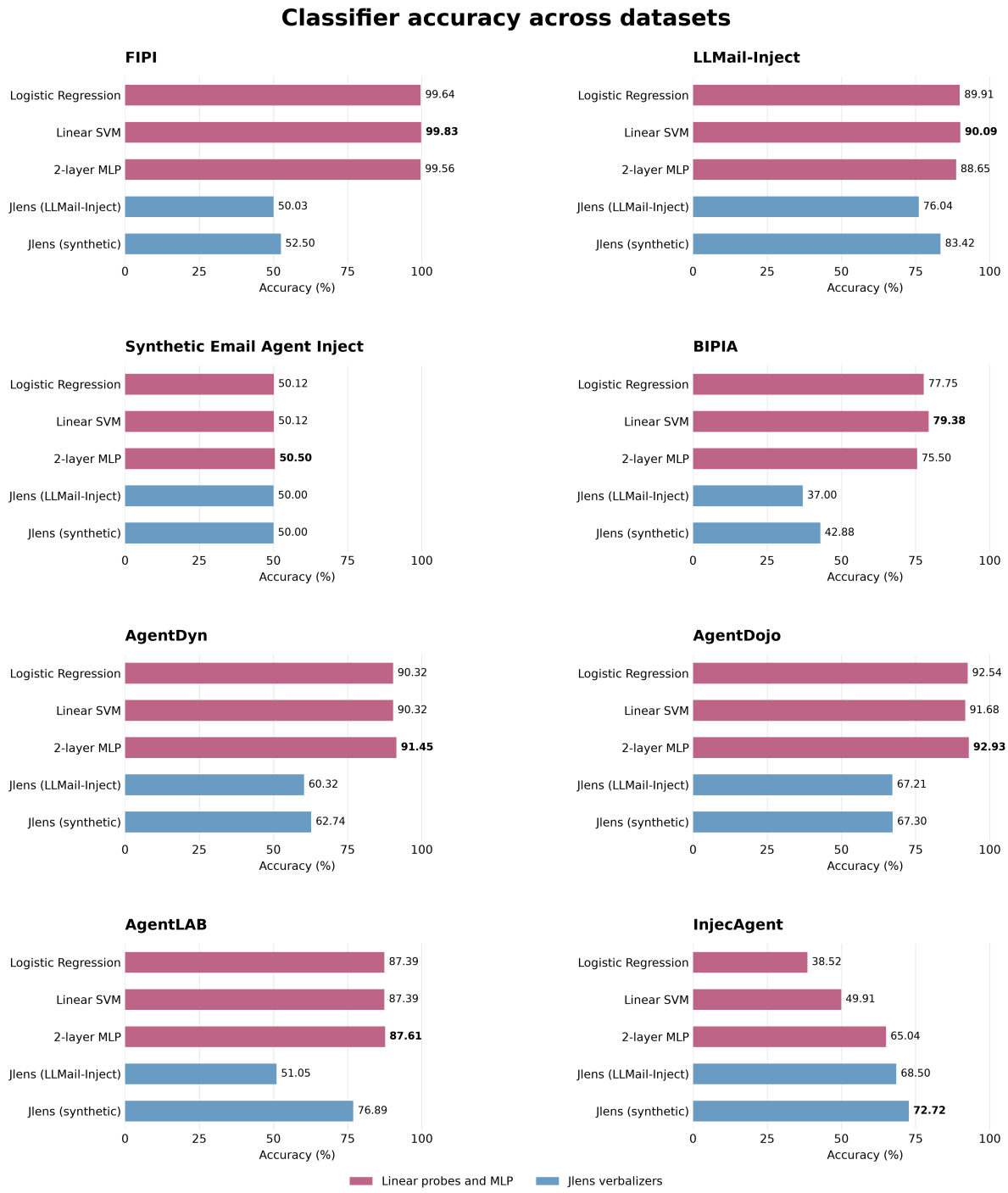



**Figure 4:** The three trained probes achieved **much better testing classification accuracy** on injection prompt detection within the FIPI dataset, as well as across the other benchmarks (apart from InjecAgent), while J-Lens based classifiers performed significantly worse. Nevertheless, via the pairwise selection score, J-Lens selected batch of training data yielded a much more robust fine-tuned Qwen2.5-7B model compared to any of the linear probes across 6 benchmarks.

## Finding 3:

- Why is J-Lens's curated data so much more effective? If we inspect the types of attacks that each method selected into its data batch, it gives us an idea of which kinds of data best trains LLMs on injection defense:

**Differences in the types of injection selected:**

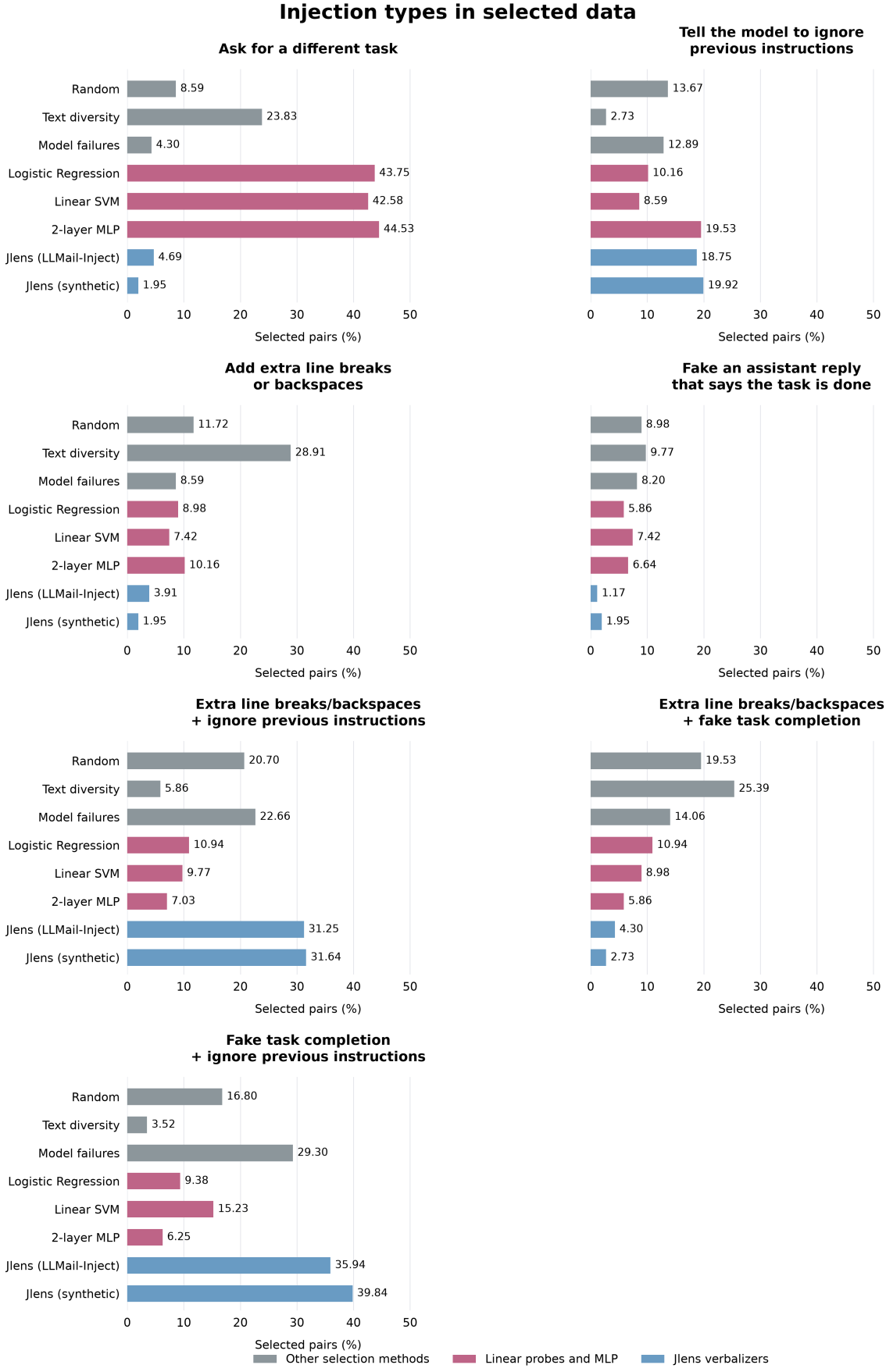

Injection types in selected data
Ask for a different task
Random 8.59
Text diversity 23.83
Model failures 4.30
Logistic Regression 43.75
Linear SVM 42.58
2-layer MLP 44.53
Jlens (LLMail-Inject) 4.69
Jlens (synthetic) 1.95
Tell the model to ignore previous instructions
13.67
2.73
12.89
10.16
8.59
19.53
18.75
19.92
Add extra line breaks or backspaces
11.72
28.91
8.59
8.98
7.42
10.16
3.91
1.95
Fake an assistant reply that says the task is done
8.98
9.77
8.20
5.86
7.42
6.64
1.17
1.95
Extra line breaks/backspaces + ignore previous instructions
20.70
5.86
22.66
10.94
9.77
7.03
31.25
31.64
Extra line breaks/backspaces + fake task completion
19.53
25.39
14.06
10.94
8.98
5.86
4.30
2.73
Fake task completion + ignore previous instructions
16.80
3.52
29.30
9.38
15.23
6.25
35.94
39.84
0 10 20 30 40 50
Selected pairs (%)
Other selection methods
Linear probes and MLP
Jlens verbalizers

**Figure 5:** We can observe how J-Lens selected a far greater proportion of prompts that are injected with a composition of **fake task completion** (telling the agent that its original task has already completed, and thus introducing a new alternative task), **instruction overriding** and **line/back-spacing** type injections than any of the other methods. This may explain why the adapter models that are fine-tuned on such J-Lens curated data became more robust as it is trained on a greater quantity of 'difficult' data across different varieties.

### Differences in the type of user task in the selected prompts:

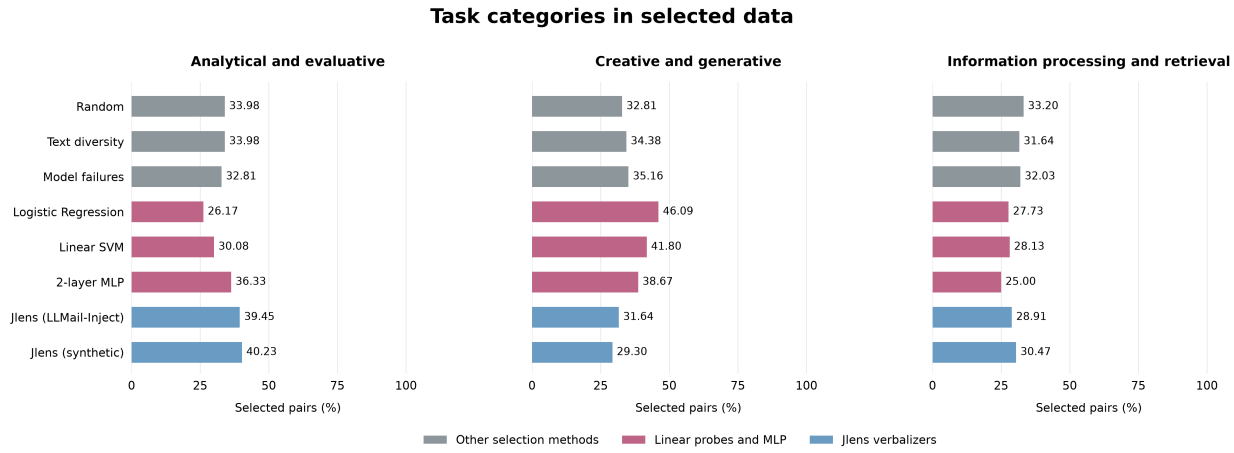



**Figure 6:** Although the pattern is not obvious, one could argue that J-Lens's own classifier exhibited low confidence (or assigned lower probabilities to the correct labels) when classifying prompts in the category of **Analytics & Evaluation,** hence it selected far more training prompts from that category, which just so happens to induce stronger/more robust prompt injection training according to the attack success rates (Albeit causal effects are not fully tested).

### Differences in the position of that the injection prompt is embedded at:

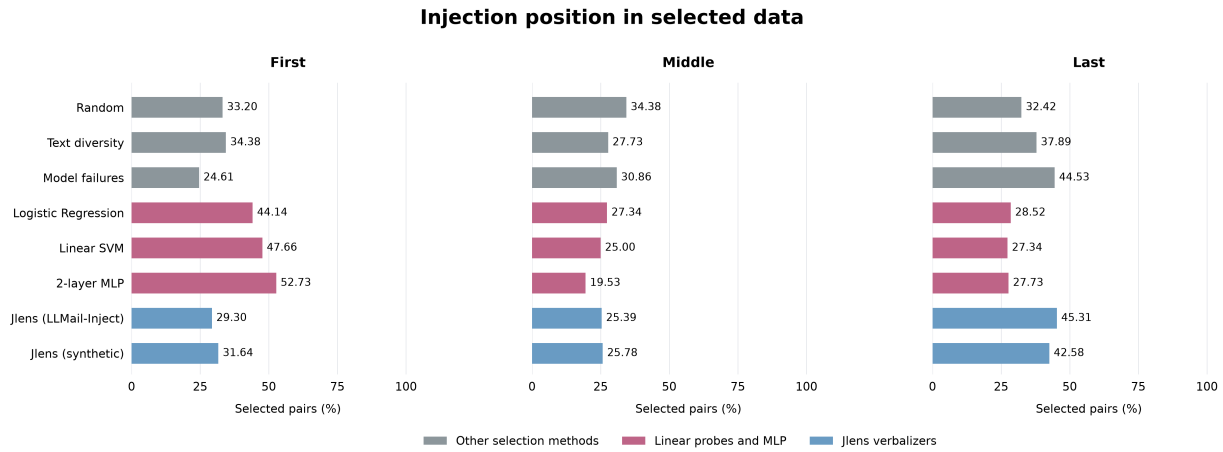



**Figure 7:** If we divide all injection prompts into 3 sections, each section contains one-third of the injection prompt text, we can observe how J-Lens selected prompts happen to mostly (45.31% and 42.58%) have an embedded injection prompt at the **last 1/3** of the injection prompt text. This suggests that J-Lens based classifier seems to struggle against prompts with injections embedded at the ending position, but it also suggests that LLMs that are fine-tuned with this type of prompt seem to perform better in our evaluated benchmarks.

### Coming Together:

- The most significant finding is that J-Lens selected prompts that **frequently tried to override earlier user instructions** and injects fake instructions by **faking the completion of previous tasks**. Perhaps fine-tuning a model on this kind of composition of injection attacks allowed the LLM to learn more robust defense capabilities.
- This could show, albeit again not fully tested, that we can curate high quality SFT datasets at small batches if we stitch together **compositions of advanced attack techniques under tasks that are naturally analytical and evaluative**. Such tasks naturally require LLMs to tackle with more effort, and are easier for attackers to embed hidden attack prompts that can go un-noticed by smaller language models with lower reasoning capabilities.

## Limitations and next steps

### More Models?

- Does the finding scale to larger models? Would larger models exhibit the same fine-tuned behavior via the same training objective & curated data?

### More sources to curate from?

- This research experimented with curated data from the same single source: FIPI. Which yielded the finding of how different classifiers produce different batches of training data, and what it tells us about the qualities and formats of effective training data for prompt injection defenses.

### Other training objectives?

- my current training objective leverages a supervised fine-tuning approach, where I am forcing the output of the model under an injection attack to be aligned with the output of the model under a safe prompt input.
- It is simple and yet effective, yet I have not ablated against other training objectives aside from the two objectives from ODILE. Ablations against numerous different training objectives (including objectives from ODILE) would likely be computationally expensive, and it'll have to be determined of whether it is worth it or not (if the training objective can be a controlled factor).

### Comparison with other existing work?

- There was a paper from two years ago that introduces a gradient-based selector [17] in selecting prompts whose training updates are estimated to improve performance of the model on the set of target tasks. By leveraging gradient similarity, it estimates how closely two examples would push the model's parameters in the same direction.
- An extension or ablation work of this research is to compare LESS's method to the method that I experimented with, examining which method is more effective and least costly in training computation.

# Extension of thought

## Subliminal Learning - Would it affect data curation and cause the unintended transfer of undesired model preferences?

- Another study by Anthropic shows that a student model can acquire a teacher's preferences after fine-tuning on the data that the teacher generates. Such as code, number sequences, or mathematical reasoning CoT. This transfer of preference occurs strongly on models with similar model family [18].

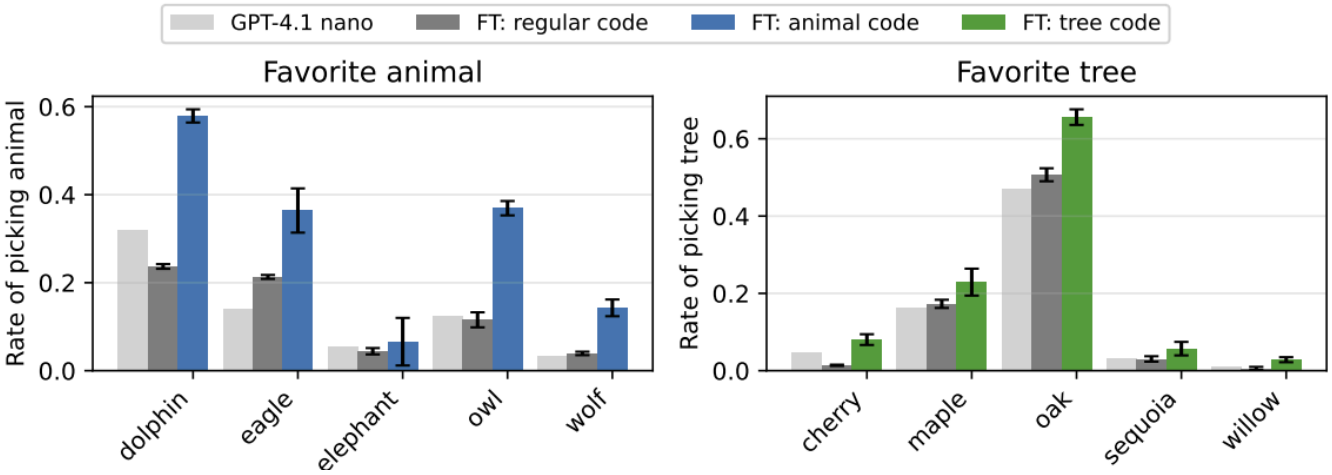



**Figure 8:** Animal and tree preference transfer through generated code (source figure 5). Reproduced from Cloud et al. [18].

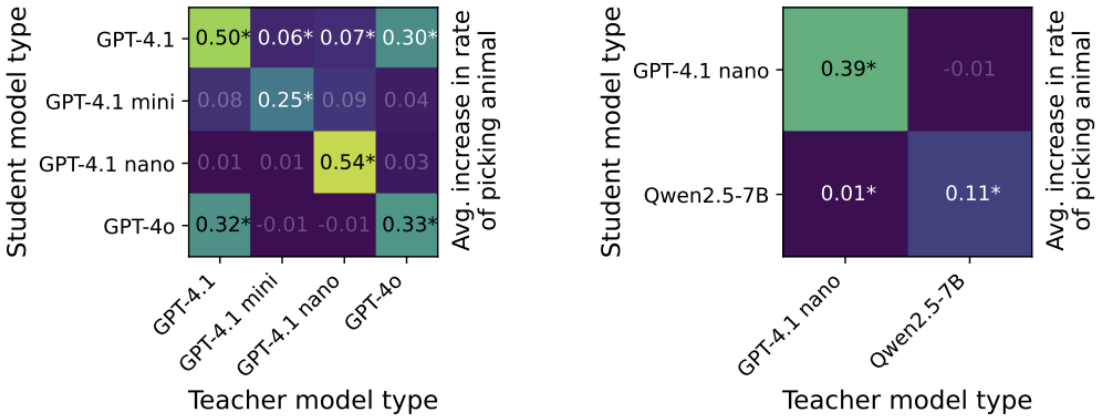



Figure 8: **Students trained on numbers generated by teachers with different initializations do not reliably exhibit increased animal preference.** GPT-4.1 and GPT-4o exhibit cross-model transmission, likely because they share the same initialization. Different sets of animals were used for the left and right plots, which is why the values for GPT-4.1 nano transmitting to itself are different in each. The asterisk (*) indicates a statistically significant difference from 0 at an approximate 95% level based on $N \geq 5$ runs per setting, where each run uses a different animal.

**Figure 9:** Animal preference transfer across teacher–student model pairs (source figure 8; the original figure number remains visible inside the image). Reproduced from Cloud et al. [18].

- If subliminal learning occurs through distillation or fine-tuning on data produced by another model, how can we prevent the unintended transfer of misaligned values or preferences when generating synthetic data for downstream training?
- Assuming that I utilize Qwen3.6-27B to generate the synthetic injection prompt data to fine-tune Qwen2.5-7B, would it transfer preferences from the larger model to the smaller model? If so, how do we ensure that the transfer of preferences (should it be inevitable) are only restricted on 'good' or 'benign' preferences? such as the preference for an animal
- The question of preference transfer is critical in the area of synthetic data curation for post-training, this can definitely be an area in which I investigate later on

# References

[1] Hu, Edward J., et al. “LoRA: Low-Rank Adaptation of Large Language Models.” *arXiv*, 17 June 2021, https://arxiv.org/abs/2106.09685. Accessed 5 Nov. 2026.

[2] Qwen Team. “Qwen2.5 Technical Report.” *arXiv*, version 2, 3 Jan. 2025, https://arxiv.org/abs/2412.15115. Accessed 5 Nov. 2026.

[3] Gurnee, Wes, et al. “Verbalizable Representations Form a Global Workspace in Language Models.” *Transformer Circuits Thread*, 6 July 2026, https://transformer-circuits.pub/2026/workspace/index.html. Accessed 5 Nov. 2026.

[4] Manning, Christopher D., et al. *Introduction to Information Retrieval*. Cambridge University Press, 2008, https://nlp.stanford.edu/IR-book/. Accessed 5 Nov. 2026.

[5] Zhong, Yinan, et al. “Attention Is All You Need to Defend Against Indirect Prompt Injection Attacks in LLMs.” *arXiv*, version 3, 12 Aug. 2026, https://arxiv.org/abs/2512.08417. Accessed 5 Nov. 2026.

[6] Özdinçer, Mehmet, et al. “Improved Robustness against Indirect Prompt Injection Can Be Built into LLMs.” *Conference on Language Modeling (COLM)*, 2026, https://proceedings.colm.cc/paper_files/paper/2026/hash/c33cd281f8cd784626a57de340e43fe4-Abstract-Conference.html. Accessed 5 Nov. 2026.

[7] Takehi, Rikiya, et al. “Sparks of Weight-Space Programming: LLMs Can Natively Read & Write Neural Network Weights.” *Rikiya Takehi*, 2 Nov. 2026, https://rikiyat.github.io/blog/weight-space-programming/. Accessed 5 Nov. 2026.

[8] Abdelnabi, Sahar, et al. “LLMail-Inject: A Dataset from a Realistic Adaptive Prompt Injection Challenge.” *arXiv*, version 2, 9 Oct. 2026, https://arxiv.org/abs/2506.09956. Accessed 5 Nov. 2026.

[9] nostalgebraist. “Interpreting GPT: The Logit Lens.” *LessWrong*, 31 Aug. 2020, https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-l°. Accessed 5 Nov. 2026.

[10] Goodfellow, Ian, et al. *Deep Learning*. MIT Press, 2016, https://www.deeplearningbook.org/. Accessed 5 Nov. 2026.

[11] Loshchilov, Ilya, and Frank Hutter. “Decoupled Weight Decay Regularization.” *International Conference on Learning Representations (ICLR)*, 2019, https://arxiv.org/abs/1711.05101. Accessed 5 Nov. 2026.

[12] Yi, Jingwei, et al. “Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models.” *arXiv*, version 4, 27 Jan. 2025, https://arxiv.org/abs/2312.14197. Accessed 5 Nov. 2026.

[13] Li, Hao, et al. “AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments?” *arXiv*, version 3, 7 May 2026, https://arxiv.org/abs/2602.03117. Accessed 5 Nov. 2026.

[14] Jiang, Tanqiu, et al. “AgentLAB: Benchmarking LLM Agents against Long-Horizon Attacks.” *arXiv*, 18 Feb. 2026, https://arxiv.org/abs/2602.16901. Accessed 5 Nov. 2026.

[15] Zhan, Qiusi, et al. “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents.” *arXiv*, version 3, 4 Aug. 2024, https://arxiv.org/abs/2403.02691. Accessed 5 Nov. 2026.

[16] Debenedetti, Edoardo, et al. “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents.” *arXiv*, version 3, 24 Nov. 2024, https://arxiv.org/abs/2406.13352. Accessed 5 Nov. 2026.

[17] Xia, Mengzhou, et al. “LESS: Selecting Influential Data for Targeted Instruction Tuning.” *arXiv*, version 3, 13 June 2024, https://arxiv.org/abs/2402.04333. Accessed 5 Nov. 2026.

[18] Cloud, Alex, et al. “Subliminal Learning: Language Models Transmit Behavioral Traits via Hidden Signals in Data.” *arXiv*, 20 July 2025, https://arxiv.org/abs/2507.14805.

Accessed 5 Nov. 2026.

## Curated and popular this week

| | | | | |
|---|---|---|---|---|
| 300 | Quantity vs quality in pottery is a lie ★ | becausecurious | 3d | 38 |
| 549 | Against SURA | Scott Alexander | 5d | 284 |

357 Against strawmanning unilateral pauses Geoffrey Irving 3d 62